SMART ELEVATOR CONTROL SYSTEM

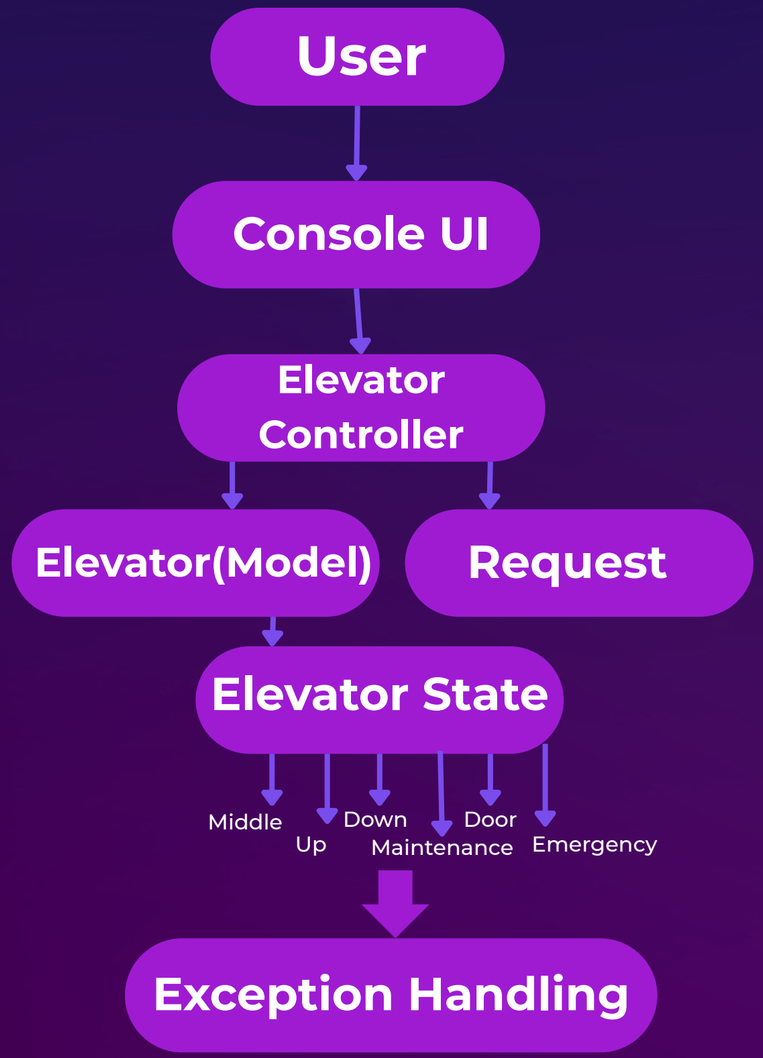
Problem Statement: To design and implement a Smart Elevator Control System that efficiently manages elevator operations using proper state management, request prioritization, and safety mechanisms such as overload detection and emergency handling.

Target Users:

- Building management systems
- Software developers & engineers
- Students & educators
- Maintenance teams

System Design

High Level Architecture



Key Classes

- Main → Entry point
- ConsoleUI → User interface
- ElevatorController → Request handling & movement
- Elevator → Core data & state
- Request → Floor request (priority supported)
- ElevatorState → Behavior interface
- States → Idle, Up, Down, Door, Emergency, Maintenance

Packages

```
src/
├── controller/
├── model/
├── state/
├── ui/
└── exception/
```

OOP Concepts Used

Abstraction

- ElevatorState interface defines common behavior

Encapsulation

- Private data in Elevator
- Access via getters/setters

Inheritance

- State classes implement ElevatorState
- Exceptions extend Exception

Polymorphism

- Same methods behave differently in each state

Design Pattern

- State Pattern → dynamic behavior change

Technical Highlights

```
while (!elevator.getCurrentState().getStateName().equals("Idle"))
 {
 elevator.getCurrentState().move(elevator);
                                        }
```

Shows polymorphism + state pattern
One line controls entire system behavior

```
if (!elevator.getPriorityRequests().isEmpty()) {
int target = elevator.getPriorityRequests().peek().getFloorNumber();
if (target > elevator.getCurrentFloor()) {
elevator.setCurrentState(new MovingUpState());
 } else {
 elevator.setCurrentState(new MovingDownState());
 }
}
```

Shows intelligence of system
Priority + dynamic direction

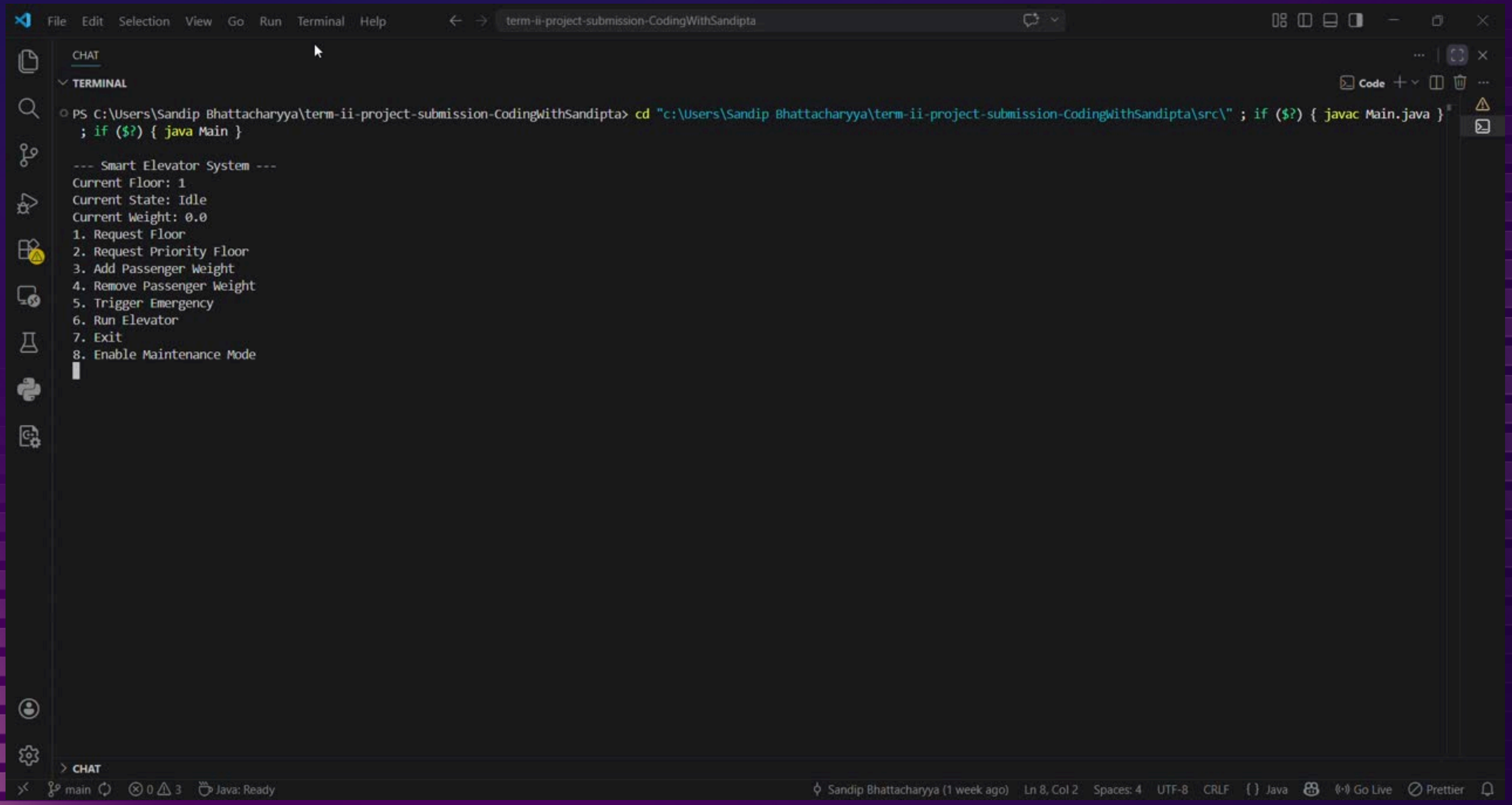
```
if (elevator.isOverloaded()) {
    throw new OverloadException("Elevator overloaded! Cannot move.");
}
```

Shows real-world safety implementation
Uses exception handling

```
System.out.println("Reached floor " + targetFloor);
elevator.setCurrentState(new DoorOpenState());
```

State Transition

DEMO


File Edit Selection View Go Run Terminal Help
term-ii-project-submission-CodingWithSandipta
CHAT
TERMINAL
Code
PS C:\Users\Sandip Bhattacharyya\term-ii-project-submission-CodingWithSandipta> cd "c:\Users\Sandip Bhattacharyya\term-ii-project-submission-CodingWithSandipta\src\" ; if ($?) { javac Main.java } ; if ($?) { java Main }
--- Smart Elevator System ---
Current Floor: 1
Current State: Idle
Current Weight: 0.0
1. Request Floor
2. Request Priority Floor
3. Add Passenger Weight
4. Remove Passenger Weight
5. Trigger Emergency
6. Run Elevator
7. Exit
8. Enable Maintenance Mode
CHAT
main
0 3
Java: Ready
Sandip Bhattacharyya (1 week ago)
Ln 8, Col 2
Spaces: 4
UTF-8
CRLF
Java
Go Live
Prettier

Thank You

Presented by : Sandipta Bhattacharyya
Stream: CSE(IOTCSBT)
Roll: 04
Enrollment No.: 12024052017021